

JAVA DSA TOOLKIT

FINAL STUDY GUIDE • JAVA-FIRST • INTERVIEW-FOCUSED

The foundation you should master before going deep into DSA patterns. Every major structure is explained through **what, why, syntax, methods, use cases, complexity, examples, pitfalls, and DSA applications**.

Layer	You learn	Outcome
01	Java syntax	Write basic DSA code without searching syntax
02	Core structures	Understand what each structure stores
03	Methods	Know what every important method does
04	Operations	Know how to manipulate each structure
05	Complexity	Know the cost of your choices
06	Decision making	Know when/where to use each tool
07	Practice	Be ready for Two Pointers and other patterns

Study rule

For every structure: **read** → **type the code yourself** → **explain each method aloud** → **reproduce the key methods from memory** → **solve tiny API exercises** → **then solve DSA problems**.

00

Roadmap

How the toolkit fits into your DSA preparation

Recommended learning order

Phase	Topics	Checkpoint
1	Java basics + complexity	Can write loops, conditions, functions and reason about $O(n)$.
2	Arrays + Strings	Can traverse, modify, search, sort and manipulate characters.
3	ArrayList + LinkedList	Can choose between fixed arrays and dynamic/list structures.
4	HashMap + HashSet	Can solve frequency, duplicate and lookup problems.
5	Stack + Queue + Deque	Can implement LIFO/FIFO and recognize their patterns.
6	PriorityQueue	Can handle repeated minimum/maximum extraction and Top-K.
7	TreeMap + TreeSet	Can answer ordered predecessor/successor/range queries.
8	Utilities + Comparator	Can sort, compare, convert and debug efficiently.
9	Pattern transition	Start Two Pointers → Sliding Window → Hashing → Stack → Binary Search...

Important

You do **not** need to memorize every method in the Java standard library. You do need to make the frequently used DSA APIs automatic.

01

Java Fundamentals

The language layer underneath every DSA solution

1.1 Variables and primitive types

Type	Definition	DSA use
int	32-bit signed integer type.	Indices, counters, normal integer values.
long	64-bit signed integer type.	Large sums, products, counts and answers.
double	64-bit floating-point type.	Problems requiring decimal calculations.
char	A UTF-16 code unit representing a character value.	Character frequency and string processing.
boolean	Stores true or false.	Flags such as found/visited.

1.2 Declaration, initialization and casting

```
int n = 10;
long total = 10_000_000_000L;
char c = 'a';
boolean found = false;

long product = (long) a * b;    // promote before multiplying
int mid = left + (right-left)/2; // safe midpoint pattern
```

Why this matters

Many DSA bugs are not algorithmic: integer overflow, integer division, wrong index bounds and accidental type conversion can invalidate a correct idea.

1.3 Control flow

```
if (condition) { ... } else { ... }

for (int i = 0; i < n; i++) { ... }

while (condition) { ... }

for (int x : arr) { ... }

break;    // immediately exits the loop
continue; // skips the rest of the current iteration
```

1.4 Methods / functions

```
static int add(int a, int b) {
    return a + b;
}

static boolean isEven(int x) {
    return x % 2 == 0;
}
```

When to use: Put reusable logic into methods so your main algorithm is easier to read, test and reason about.

1.5 Operators

Category	Operators	Typical DSA role
Arithmetic	+ - * / %	sum, difference, parity, modulo
Comparison	< <= > >= == !=	bounds and state checks
Logical	&& !	combine constraints
Ternary	condition ? a : b	compact conditional value
Bitwise	& ^ ~ << >> >>>	bit manipulation and masks

02

Arrays

Fixed-size indexed storage and the foundation of many patterns

2.1 What is an array?

An array stores multiple values of the same type in a fixed-length structure where each element is accessed by an integer index. In Java, indexing starts at 0.

2.2 Declaration and creation

```
int[] a; // declaration
int[] b = new int[5]; // creation, default values are 0
int[] c = {10, 20, 30, 40}; // declaration + initialization
int[] d = new int[]{1, 2, 3};

int[][] matrix = new int[3][4]; // 2D array
```

2.3 Core properties

- **Fixed length:** `a.length` never changes after creation.
- **Zero indexed:** first element is `a[0]`.
- **O(1) access:** `a[i]` directly reaches an element.
- **Homogeneous:** all elements have the same declared type.
- **Default initialization:** `int` → 0, `boolean` → `false`, `reference` → `null`.

2.4 Common operations

Operation	Definition	Typical time	Use
Access	Read a value using its index.	O(1)	Random access.
Update	Replace a value at an index.	O(1)	In-place changes.
Linear search	Scan elements until a match is found.	O(n)	Unsorted lookup.
Binary search	Repeatedly halve a sorted search range.	O(log n)	Sorted lookup.
Insert/delete	Shift elements or build a new array.	O(n)	Manual fixed-array manipulation.
Sort	Rearrange elements into order.	O(n log n) typical	Prepare for many patterns.

2.5 Traversal templates

```
for (int i = 0; i < a.length; i++) {
    int value = a[i];
}

for (int value : a) {
    // value is the current element
}

for (int i = a.length - 1; i >= 0; i--) {
    // reverse traversal
}
```

2.6 Arrays utility methods — defined individually

Method	One-line definition	When / where to use
<code>Arrays.sort(a)</code>	Sorts the entire array into ascending natural order.	Before two pointers, binary search or ordered processing.
<code>Arrays.sort(a,l,r)</code>	Sorts the range <code>[l,r)</code> , where <code>r</code> is exclusive.	Sort only a subrange.

Method	One-line definition	When / where to use
<code>Arrays.binarySearch(a,x)</code>	Searches for x in a sorted array and returns its index if found.	Fast lookup when data is sorted.
<code>Arrays.fill(a,x)</code>	Assigns x to every element or a selected range.	Reset/init arrays such as visited or DP arrays.
<code>Arrays.copyOf(a,n)</code>	Creates a new array of length n containing copied elements.	Resize or make a copy.
<code>Arrays.copyOfRange(a,l,r)</code>	Creates a new array containing elements in [l,r).	Extract a subarray.
<code>Arrays.equals(a,b)</code>	Checks whether two 1D arrays contain equal elements in the same order.	Compare arrays.
<code>Arrays.toString(a)</code>	Returns a readable string representation of a 1D array.	Debugging / output.
<code>Arrays.deepToString(a)</code>	Returns a readable representation of nested arrays.	Debugging matrices.

Pitfall

`Arrays.binarySearch` expects the relevant range to be sorted. Also, its negative return value when an element is absent is not a normal array index.

03

2D Arrays & Matrices

Grid traversal, boundaries and neighbor templates

3.1 Declaration

```
int[][] grid = new int[rows][cols];
int[][] m = {{1,2,3}, {4,5,6}};

int rows = m.length;
int cols = m[0].length; // only if the matrix is non-empty
```

3.2 Traversal

```
for (int r = 0; r < grid.length; r++) {
    for (int c = 0; c < grid[r].length; c++) {
        int value = grid[r][c];
    }
}
```

3.3 Four-direction neighbor template

```
int[] dr = {-1, 1, 0, 0};
int[] dc = {0, 0, -1, 1};

for (int k = 0; k < 4; k++) {
    int nr = r + dr[k];
    int nc = c + dc[k];

    if (nr >= 0 && nr < grid.length &&
        nc >= 0 && nc < grid[nr].length) {
        // process neighbor
    }
}
```

Where used: matrix traversal, island/grid problems, BFS/DFS, shortest paths on grids, boundary simulation and dynamic programming.

Jagged arrays

Java allows each row to have a different length. Therefore `grid[r].length` is safer than assuming every row has the same number of columns.

04

String

Immutable text and character-processing toolkit

4.1 What is String?

String is Java's immutable text type. Once a String object exists, its characters cannot be changed in place; operations that appear to modify it create another String.

4.2 Creation

```
String s1 = "hello";
String s2 = new String("hello"); // valid, but usually unnecessary
String empty = "";
```

4.3 String methods — defined individually

Method	Definition	Typical DSA use
length()	Returns the number of UTF-16 code units in the string.	Loop bounds and size checks.
charAt(i)	Returns the character at index i.	Character scanning.
substring(l,r)	Returns a new string containing [l,r).	Extract windows/ranges.
equals(x)	Checks content equality with another string.	Correct string comparison.
equalsIgnoreCase(x)	Checks content equality ignoring case.	Case-insensitive problems.
compareTo(x)	Compares two strings lexicographically.	Sorting / dictionary order.
indexOf(x)	Returns the first occurrence index, or -1 if absent.	Search for character/subsequence.
lastIndexOf(x)	Returns the last occurrence index, or -1 if absent.	Find latest occurrence.
contains(x)	Checks whether x occurs as a substring.	Existence checks.
startsWith(x)	Checks whether the string begins with x.	Prefix checks.
endsWith(x)	Checks whether the string ends with x.	Suffix checks.
toCharArray()	Copies the string characters into a char array.	In-place-like character processing.
split(regex)	Splits the string around matches of a regular expression.	Tokenization when needed.
replace(a,b)	Replaces literal character/sequence occurrences.	Simple transformations.
replaceAll(regex,r)	Replaces matches of a regular expression.	Regex-based transformations.
trim()	Removes leading/trailing characters <= U+0020.	Basic input cleanup; not general Unicode whitespace.

4.4 String comparison — critical

```
String a = "cat";
String b = "cat";

a.equals(b); // content comparison
a == b;      // reference comparison – NOT the normal choice
```

Rule to memorize

For DSA string-content comparison, use **equals()** unless you specifically need reference identity.

4.5 Character utilities

Method	Definition	Use
Character.isDigit(c)	Checks whether c is a digit character.	Parsing / numeric-character problems.
Character.isLetter(c)	Checks whether c is a letter.	Validation / filtering.

Method	Definition	Use
Character.isLetterOrDigit(c)	Checks whether c is a letter or digit.	Alphanumeric filtering.
Character.toLowerCase(c)	Returns the lowercase form of c.	Normalize before comparison.
Character.toUpperCase(c)	Returns the uppercase form of c.	Normalize output/comparison.

05

StringBuilder

Mutable strings for efficient construction and reversal

5.1 What is it?

StringBuilder is a mutable sequence of characters. It is useful when repeated concatenation or character edits would otherwise create many temporary String objects.

5.2 Core methods

Method	Definition	When to use
append(x)	Adds x to the end.	Build output incrementally.
insert(i,x)	Inserts x at index i.	Insert into mutable text.
delete(l,r)	Removes characters in [l,r).	Delete a range.
deleteCharAt(i)	Removes the character at i.	Single-character deletion.
setCharAt(i,c)	Replaces the character at i.	Mutable character update.
charAt(i)	Returns the character at i.	Inspect characters.
length()	Returns the current character count.	Bounds / state.
reverse()	Reverses the sequence in place.	Palindrome/reversal operations.
toString()	Creates a String containing the current sequence.	Final result.

```
StringBuilder sb = new StringBuilder();
sb.append("Hello");
sb.append(' ');
sb.append("World");

sb.setCharAt(0, 'h');
sb.deleteCharAt(5);
sb.reverse();

String answer = sb.toString();
```

Where used

Output construction, string reversal, backtracking-generated strings, parsing and any problem involving many repeated concatenations.

06

ArrayList / List

Dynamic indexed sequence backed by a resizable array

6.1 What is ArrayList?

ArrayList is a resizable-array implementation of the List interface. It keeps insertion order, permits duplicates and provides fast indexed access.

6.2 Declaration

```
List<Integer> list = new ArrayList<>();
ArrayList<String> names = new ArrayList<>();

list.add(10);
list.add(20);
```

6.3 Methods — definition + use

Method	Definition	Typical complexity	Use
add(x)	Appends x to the end.	O(1) amortized	Build a dynamic sequence.
add(i,x)	Inserts x at index i and shifts later elements.	O(n)	Insert into a position.
get(i)	Returns the element at index i.	O(1)	Random access.
set(i,x)	Replaces the element at i and returns the old value.	O(1)	Update.
remove(i)	Removes the element at index i and shifts later elements.	O(n)	Delete by index.
remove(Object)	Removes the first matching object, if present.	O(n)	Delete by value.
contains(x)	Checks whether an equal element exists.	O(n)	Membership test.
indexOf(x)	Returns first matching index or -1.	O(n)	Find position.
lastIndexOf(x)	Returns last matching index or -1.	O(n)	Find latest position.
size()	Returns number of stored elements.	O(1)	Loop bounds.
isEmpty()	Returns true if size is zero.	O(1)	State check.
clear()	Removes all elements.	O(n)	Reset the list.
subList(l,r)	Returns a view of [l,r), not an independent copy.	O(1) creation	Work with a range carefully.

6.4 The remove(Integer) trap

```
List<Integer> a = new ArrayList<>(List.of(10, 20, 30));

a.remove(1);           // removes index 1 → 20
a.remove(Integer.valueOf(30)); // removes value 30
```

Memorize this

For List, remove(1) means **index**, while remove(Integer.valueOf(1)) means **value**.

07

LinkedList

List + Deque behavior, plus the custom Node model used in interviews

7.1 Java LinkedList

LinkedList is a doubly linked list implementation that also implements Deque. It supports efficient insertion/removal at its ends, but indexed access is $O(n)$.

```
LinkedList<Integer> list = new LinkedList<>();

list.addFirst(10);
list.addLast(20);
list.getFirst();
list.getLast();
list.removeFirst();
list.removeLast();
```

Method	Definition	Use
addFirst(x)	Adds x to the front.	Deque/list front insertion.
addLast(x)	Adds x to the end.	End insertion.
getFirst()	Returns the first element without removing it.	Inspect front.
getLast()	Returns the last element without removing it.	Inspect back.
removeFirst()	Removes and returns the first element.	Queue/deque behavior.
removeLast()	Removes and returns the last element.	Stack/deque behavior.

7.2 Custom linked-list node

```
class Node {
    int val;
    Node next;

    Node(int val) {
        this.val = val;
    }
}
```

Interview linked-list problems usually give you nodes rather than Java's LinkedList class. You therefore need to understand references: **node.next** points to the next node.

7.3 Essential linked-list ideas

- Dummy/sentinel node simplifies insertion/deletion at the head.
- Two references such as slow/fast are used to find a middle or detect a cycle.
- Reversal changes next references one link at a time.
- Merging sorted lists compares current nodes and advances one pointer.
- Always protect against null before dereferencing node.next.

08

HashMap

Fast average key → value lookup: the workhorse of DSA

8.1 What is HashMap?

HashMap stores key-value pairs. Keys are unique: inserting an existing key replaces its old value. Typical get/put/remove operations are $O(1)$ on average.

8.2 Declaration

```
Map<String, Integer> map = new HashMap<>();
Map<Integer, String> names = new HashMap<>();

map.put("apple", 3);
map.put("apple", 5);           // updates the value
```

8.3 Methods — definition + use

Method	Definition	Typical use
put(k,v)	Associates key k with value v; replaces an existing value.	Store/update mapping.
get(k)	Returns the value for k, or null if no mapping exists.	Lookup.
getOrDefault(k,d)	Returns mapped value, or d when k is absent.	Frequency counting.
containsKey(k)	Checks whether k is present as a key.	Existence check.
containsValue(v)	Checks whether some key maps to v.	Rare in DSA; $O(n)$.
remove(k)	Removes k and returns its previous value, if any.	Delete mapping.
putIfAbsent(k,v)	Adds mapping only if k has no value.	Initialize once.
replace(k,v)	Replaces value only when the key exists.	Conditional update.
keySet()	Returns a view of all keys.	Iterate keys.
values()	Returns a view of all values.	Iterate values.
entrySet()	Returns key-value entries for efficient pair iteration.	Iterate both key and value.

8.4 Frequency-map template

```
Map<Integer, Integer> freq = new HashMap<>();

for (int x : arr) {
    freq.put(x, freq.getOrDefault(x, 0) + 1);
}
```

8.5 Grouping template

```
Map<Character, List<Integer>> positions = new HashMap<>();

positions.computeIfAbsent(c, k -> new ArrayList<>())
    .add(index);
```

When to use HashMap

Use it when a problem asks for **count**, **frequency**, **lookup**, **complement**, **mapping**, **grouping**, **first/last index**, or **seen-state keyed by a value**.

09

HashSet

Unique values + fast average membership

9.1 What is HashSet?

HashSet stores unique elements. Adding an element already present does nothing. It is ideal when you care about membership or uniqueness, not key-value mappings.

```
Set<Integer> seen = new HashSet<>();

seen.add(10);
seen.add(20);
seen.add(10);           // duplicate ignored

boolean exists = seen.contains(20);
seen.remove(10);
```

Method	Definition	Typical use
add(x)	Adds x if it is not already present.	Build unique set.
contains(x)	Checks whether x is present.	O(1) average membership.
remove(x)	Removes x if present.	Maintain active set.
size()	Returns number of unique elements.	Count unique values.
isEmpty()	Checks whether the set has no elements.	State check.
clear()	Removes all elements.	Reset.

Common DSA uses

- Duplicate detection.
- Visited values/nodes.
- Membership checks inside a linear scan.
- Distinct-element counting.
- Checking whether a complement/value has already appeared.

HashSet vs HashMap

If you only need “**have I seen this?**”, use HashSet. If you need “**what information is associated with this key?**”, use HashMap.

10

Stack • Queue • Deque

Three interfaces you will repeatedly meet in DSA patterns

10.1 Stack using ArrayDeque

A stack follows LIFO: Last In, First Out. In modern Java DSA code, ArrayDeque is generally preferred over the legacy Stack class.

```
Deque<Integer> stack = new ArrayDeque<>();

stack.push(10);
stack.push(20);

int top = stack.peek(); // 20
int x = stack.pop();    // 20
```

Method	Definition	Use
push(x)	Adds x to the front/top of the deque.	Stack insertion.
peek()	Returns top without removing it.	Inspect top.
pop()	Removes and returns top.	Stack removal.
isEmpty()	Checks whether stack has no elements.	Loop/termination.

10.2 Queue using ArrayDeque

A queue follows FIFO: First In, First Out. Use offer to enqueue and poll to dequeue.

```
Queue<Integer> q = new ArrayDeque<>();

q.offer(10);
q.offer(20);

int front = q.peek(); // 10
int x = q.poll();     // 10
```

Method	Definition	Use
offer(x)	Adds x to the queue when possible.	Enqueue.
peek()	Returns front without removing it.	Inspect front.
poll()	Removes and returns front, or null if empty.	Dequeue.

10.3 Deque

Deque means double-ended queue. It supports insertion/removal from both ends and is extremely important for sliding-window and monotonic-queue patterns.

```
Deque<Integer> dq = new ArrayDeque<>();

dq.offerFirst(1);
dq.offerLast(2);

dq.peekFirst();
dq.peekLast();

dq.pollFirst();
dq.pollLast();
```

11

PriorityQueue / Heap

Repeatedly retrieve the smallest or largest priority element

11.1 What is PriorityQueue?

PriorityQueue is a heap-backed queue. Java's default PriorityQueue is a **min-heap**: `peek()` and `poll()` expose the smallest element according to the queue's ordering.

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

minHeap.offer(5);
minHeap.offer(1);
minHeap.offer(3);

int smallest = minHeap.peek(); // 1
int removed = minHeap.poll(); // 1
```

11.2 Max-heap

```
PriorityQueue<Integer> maxHeap =
    new PriorityQueue<>(Comparator.reverseOrder());
```

11.3 Methods

Method	Definition	Typical complexity	Use
<code>offer(x)</code>	Adds x while maintaining heap order.	$O(\log n)$	Insert candidate.
<code>add(x)</code>	Adds x; for PriorityQueue commonly equivalent to <code>offer</code> .	$O(\log n)$	Alternative insertion.
<code>peek()</code>	Returns highest-priority element without removing it.	$O(1)$	Inspect min/max.
<code>poll()</code>	Removes and returns highest-priority element.	$O(\log n)$	Repeated extraction.
<code>isEmpty()</code>	Checks whether heap has no elements.	$O(1)$	Termination.
<code>size()</code>	Returns number of elements.	$O(1)$	State.

11.4 Custom priority

```
PriorityQueue<int[]> pq =
    new PriorityQueue<>((a, b) ->
        Integer.compare(a[0], b[0]));
```

Use cases: Top-K, K-way merge, scheduling, Dijkstra-like algorithms, continuously selecting the next smallest/largest candidate.

Important

A PriorityQueue is **not fully sorted** when iterated. Only the head is guaranteed to be the highest-priority element.

12

TreeMap & TreeSet

Ordered structures for predecessor, successor and range queries

12.1 TreeMap

TreeMap stores key-value pairs in sorted key order. Typical search/insert/remove operations are $O(\log n)$.

```
TreeMap<Integer, String> map = new TreeMap<>();

map.put(10, "A");
map.put(5, "B");
map.put(20, "C");

map.firstKey();
map.lastKey();
map.floorKey(12);
map.ceilingKey(12);
map.lowerKey(10);
map.higherKey(10);
```

Method	Definition	Use
firstKey()	Returns the smallest key.	Minimum key.
lastKey()	Returns the largest key.	Maximum key.
floorKey(k)	Returns greatest key $\leq k$, or null.	Predecessor including k.
ceilingKey(k)	Returns smallest key $\geq k$, or null.	Successor including k.
lowerKey(k)	Returns greatest key $< k$.	Strict predecessor.
higherKey(k)	Returns smallest key $> k$.	Strict successor.
firstEntry()/lastEntry()	Returns smallest/largest key-value entry.	Inspect extremes.
pollFirstEntry()/pollLastEntry()	Removes and returns extreme entry.	Consume ordered extremes.

12.2 TreeSet

TreeSet stores unique elements in sorted order. It provides the same style of floor/ceiling/lower/higher operations without values attached to keys.

```
TreeSet<Integer> set = new TreeSet<>();

set.add(10);
set.add(5);
set.add(20);

set.first();
set.last();
set.floor(12);
set.ceiling(12);
set.lower(10);
set.higher(10);
```

When to use

Choose TreeMap/TreeSet when **order itself matters**: nearest smaller/larger, predecessor/successor, sorted unique values, or range queries.

13

Collections Utility Class

Algorithms and helpers for collection objects

13.1 Important methods

Method	Definition	Use
<code>Collections.sort(list)</code>	Sorts a List using its natural ordering or supplied comparator.	Sort lists.
<code>Collections.reverse(list)</code>	Reverses the list in place.	Reverse order.
<code>Collections.swap(list,i,j)</code>	Exchanges two list positions.	Manual rearrangement.
<code>Collections.min(list)</code>	Returns the smallest element according to natural ordering.	Find minimum.
<code>Collections.max(list)</code>	Returns the largest element according to natural ordering.	Find maximum.
<code>Collections.frequency(list,x)</code>	Counts how many elements equal x.	Simple frequency checks.
<code>Collections.binarySearch(list,x)</code>	Binary-searches a sorted list.	Fast lookup in sorted list.
<code>Collections.fill(list,x)</code>	Replaces every list element with x.	Reset list values.

13.2 Arrays vs Collections

Tool	Works on	Remember
Arrays	Java arrays	<code>Arrays.sort</code> , <code>binarySearch</code> , <code>fill</code> , <code>copyOf</code> , <code>equals</code> .
Collections	Collection/List objects	<code>sort</code> , <code>reverse</code> , <code>swap</code> , <code>min</code> , <code>max</code> , <code>frequency</code> .
Collection	Interface family	Not a utility class.
Collections	Utility class	Static helper methods.

14

Comparable & Comparator

Control ordering for sorting and heaps

14.1 Comparable

Comparable defines a class's natural ordering by implementing compareTo.

```
class Person implements Comparable<Person> {
    int age;

    public int compareTo(Person other) {
        return Integer.compare(this.age, other.age);
    }
}
```

14.2 Comparator

Comparator is a separate comparison strategy. It is ideal when you need different orderings for the same class or cannot modify the class.

```
Comparator<Person> byAge =
    Comparator.comparingInt(p -> p.age);

Comparator<Person> byAgeDesc =
    byAge.reversed();

list.sort(byAge);
```

14.3 Useful comparator methods

Method	Definition	Use
Comparator.comparing(...)	Builds a comparator from a key-extraction function.	Sort objects by a field.
Comparator.comparingInt(...)	Builds a comparator for an int-valued key.	Efficient int-field ordering.
reversed()	Returns the reverse ordering.	Descending sort.
thenComparing(...)	Adds a tie-breaker comparator.	Sort by multiple fields.
Integer.compare(a,b)	Safely compares two ints without subtraction overflow.	Comparator implementation.

```
list.sort(
    Comparator.comparingInt((Person p) -> p.age)
                .thenComparing(p -> p.name)
);
```

Comparator pitfall

Avoid **a - b** as a general integer comparator because subtraction can overflow. Prefer `Integer.compare(a,b)`.

15

Conversions & Parsing

Moving cleanly between common Java representations

15.1 Array → List

```
Integer[] boxed = {1, 2, 3};  
List<Integer> list = new ArrayList<>(Arrays.asList(boxed));
```

Arrays.asList on an object array creates a fixed-size list view; wrapping it in new ArrayList gives a resizable list.

15.2 List → array

```
Integer[] arr = list.toArray(new Integer[0]);
```

15.3 Primitive int[] → List

```
int[] a = {1, 2, 3};  
  
List<Integer> list = Arrays.stream(a)  
    .boxed()  
    .toList();
```

Note: toList() produces an unmodifiable list in modern Java. If you need a mutable ArrayList, wrap it: **new ArrayList<>(...)**.

15.4 String ↔ char[]

```
char[] chars = s.toCharArray();  
String s2 = new String(chars);
```

15.5 Parsing

Method	Definition	Use
Integer.parseInt(s)	Parses a String into an int.	Input/string-to-number conversion.
Long.parseLong(s)	Parses a String into a long.	Large numeric conversion.
String.valueOf(x)	Returns the string representation of x.	Number/object → String.

Primitive vs wrapper

int and Integer are different types. DSA collections such as List store Integer objects, not primitive int values.

16

Math & General Utilities

Small methods that repeatedly appear inside algorithms

16.1 Math methods

Method	Definition	Use
Math.max(a,b)	Returns the larger value.	Bounds / running maximum.
Math.min(a,b)	Returns the smaller value.	Bounds / running minimum.
Math.abs(x)	Returns absolute magnitude.	Distance/difference calculations.
Math.pow(a,b)	Returns a floating-point power result.	Power calculations; cast carefully if needed.
Math.sqrt(x)	Returns square root.	Geometry / number problems.
Math.floor(x)	Rounds toward negative infinity.	Floating-point calculations.
Math.ceil(x)	Rounds toward positive infinity.	Floating-point calculations.

16.2 Useful constants

```
Integer.MAX_VALUE;  
Integer.MIN_VALUE;  
Long.MAX_VALUE;  
Long.MIN_VALUE;
```

16.3 Common DSA helpers

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int temp = a % b;  
        a = b;  
        b = temp;  
    }  
    return Math.abs(a);  
}  
  
boolean isEven(int x) {  
    return x % 2 == 0;  
}
```

17

Iteration Patterns

How to safely walk through the structures

17.1 Lists and arrays

```
for (int i = 0; i < list.size(); i++) {
    int x = list.get(i);
}

for (int x : list) {
    // process x
}
```

For ArrayList, indexed iteration is normally efficient. For LinkedList, repeated `get(i)` can make a loop $O(n^2)$; prefer iterator/enhanced-for traversal.

17.2 Maps

```
for (Integer key : map.keySet()) {
    int value = map.get(key);
}

for (int value : map.values()) {
    // process value
}

for (Map.Entry<Integer,Integer> e : map.entrySet()) {
    int key = e.getKey();
    int value = e.getValue();
}
```

Best map iteration

When you need both key and value, **`entrySet()`** is the clearest and usually the preferred approach.

17.3 Safe removal

```
Iterator<Integer> it = list.iterator();

while (it.hasNext()) {
    int x = it.next();

    if (x < 0) {
        it.remove();
    }
}
```

Do not structurally modify a collection inside an enhanced for-loop unless the operation is specifically supported. `Iterator.remove` is the standard safe pattern.

18

Complexity Reference

Know the cost of the operation before choosing the structure

18.1 Core complexity table

Structure / operation	Typical time	Why you care
Array access/update	$O(1)$	Fast indexed work.
Array linear search	$O(n)$	Unsorted lookup.
ArrayList get/set	$O(1)$	Fast indexed access.
ArrayList add(end)	$O(1)$ amortized	Good dynamic append.
ArrayList add/remove middle	$O(n)$	Shifting is required.
HashMap get/put/remove	$O(1)$ average	Fast keyed lookup.
HashSet add/contains/remove	$O(1)$ average	Fast membership.
LinkedList end insert/remove	$O(1)$	Good when position/node is known.
LinkedList get(i)	$O(n)$	No random-access advantage.
ArrayDeque end operations	$O(1)$ amortized	Excellent stack/queue/deque.
PriorityQueue peek	$O(1)$	Fast best-priority access.
PriorityQueue offer/poll	$O(\log n)$	Heap maintenance.
TreeMap/TreeSet operations	$O(\log n)$	Ordered search/update.

18.2 Big-O reflexes

- One full loop over n elements $\rightarrow$ usually $O(n)$.
- Nested independent loops $\rightarrow$ often $O(n^2)$.
- Halving search space repeatedly $\rightarrow O(\log n)$.
- Sort + linear scan $\rightarrow$ usually $O(n \log n)$.
- HashMap/HashSet operations $\rightarrow O(1)$ average, not a mathematical worst-case guarantee.
- Always account for auxiliary space when the problem asks for optimal complexity.

19

Which Tool Should I Choose?

Translate problem wording into a Java structure

Problem clue	First structure to consider	Reason
Need index-based random access	array / ArrayList	O(1) access.
Fixed-size numeric data	array	Simple + memory efficient.
Dynamic sequence + random access	ArrayList	Resizable array.
Need unique values	HashSet	Membership + uniqueness.
Need value → information	HashMap	Key-value mapping.
Need sorted unique values	TreeSet	Order + uniqueness.
Need sorted key → value	TreeMap	Order + mapping.
Repeated smallest/largest	PriorityQueue	Heap extraction.
Last-in-first-out	ArrayDeque as stack	LIFO operations.
First-in-first-out	ArrayDeque as queue	FIFO operations.
Both ends	ArrayDeque	Double-ended operations.
Mutable string construction	StringBuilder	Avoid repeated immutable concatenation.
Custom sorting	Comparator	Define ordering.

Recognition examples

If you see...	Think...
"frequency / count occurrences"	HashMap
"have we seen this?"	HashSet
"two values sum to target"	HashMap or sorted array + Two Pointers
"next greater element"	Stack / monotonic stack
"top K"	PriorityQueue
"closest smaller/larger"	TreeSet / TreeMap or sorted + binary search
"continuous range / substring"	Often Sliding Window + HashMap/Set
"sorted array + pair"	Often Two Pointers

20

Java DSA Pitfalls

Small language mistakes that cause big algorithm bugs

Pitfall	What goes wrong	Correct habit
String == String	Compares references, not normal content.	Use equals().
Integer division	5/2 becomes 2.	Use double when decimal result is needed.
int overflow	Large sums/products wrap around.	Use long and promote early.
List.remove(1)	Removes index 1, not value 1.	Use remove(Integer.valueOf(1)) for value.
Binary search on unsorted data	Result is meaningless.	Sort or maintain required ordering.
ArrayList middle insertion	Shifts elements.	Remember O(n).
LinkedList get(i) in loop	Repeated traversal can become O(n ²).	Traverse with node references/iterator.
PriorityQueue iteration	Does not output sorted order.	Repeatedly poll if sorted extraction is needed.
Comparator subtraction	a-b can overflow.	Use Integer.compare / Long.compare.
Off-by-one range	substring/copyOfRange use exclusive end.	Think [left,right).
Null dereference	Calling a method on null crashes.	Check references before dereference.
Modifying collection in for-each	May cause ConcurrentModificationException.	Use iterator or controlled index logic.

21

Must-Know Templates

The small code shapes you should be able to write from memory

Array traversal + min/max

```
int min = Integer.MAX_VALUE;
int max = Integer.MIN_VALUE;

for (int x : arr) {
    min = Math.min(min, x);
    max = Math.max(max, x);
}
```

Frequency map

```
Map<Integer,Integer> freq = new HashMap<>();

for (int x : arr) {
    freq.put(x, freq.getOrDefault(x, 0) + 1);
}
```

Seen set

```
Set<Integer> seen = new HashSet<>();

for (int x : arr) {
    if (seen.contains(x)) {
        // duplicate
    }
    seen.add(x);
}
```

Stack

```
Deque<Integer> stack = new ArrayDeque<>();

for (int x : arr) {
    while (!stack.isEmpty() && stack.peek() < x) {
        stack.pop();
    }
    stack.push(x);
}
```

Queue

```
Queue<Integer> q = new ArrayDeque<>();

q.offer(start);

while (!q.isEmpty()) {
    int cur = q.poll();
    // process cur
}
```

PriorityQueue Top-K skeleton

```
PriorityQueue<Integer> pq = new PriorityQueue<>();

for (int x : arr) {
    pq.offer(x);
    if (pq.size() > k) {
        pq.poll();
    }
}
```

A. Syntax

- I can declare and initialize arrays, Lists, Sets, Maps, Deques and heaps.
- I can write for, while and enhanced-for loops without looking up syntax.
- I know int vs long and understand integer division.
- I can write small helper methods.

B. Arrays / Strings

- I know array length vs List size().
- I can traverse forward/backward and handle boundaries.
- I know Arrays.sort, binarySearch, fill, copyOf, copyOfRange and equals.
- I know String methods and use equals for content comparison.
- I can use StringBuilder for mutable construction.

C. Collections

- I can explain ArrayList get/add/remove/set and their costs.
- I know the remove(index) vs remove(value) Integer trap.
- I can build a frequency HashMap from memory.
- I can use HashSet for membership/duplicates.
- I can implement stack, queue and deque behavior with ArrayDeque.
- I can create min-heaps and max-heaps with PriorityQueue.
- I know when TreeMap/TreeSet is preferable.

D. Ordering / utilities

- I can sort with Comparator.
- I know comparingInt, reversed and thenComparing.
- I use Integer.compare instead of subtraction comparators.
- I know common Collections and Math utilities.
- I can convert between arrays, lists, strings and char arrays.

E. Decision making

- Given a problem, I can name the likely data structure before coding.
- I can state the expected time and space complexity of my chosen operations.
- I can identify when a sorted structure, hash structure, heap, stack or deque is appropriate.

Ready for the next layer?

If these are mostly "yes", your next study unit should be **TWO POINTERS**. Break it into: recognition → pointer movement rules → variants → templates → simple problems → medium problems → LeetCode-level problems.

23

7-Day Java Toolkit Plan

A practical first pass before starting DSA patterns

Day	Study	Practice output
1	Java basics + arrays	Write declarations, traversal, min/max, reverse, search and Arrays utilities from memory.
2	2D arrays + String	Matrix traversal + 10 String method examples + character processing.
3	StringBuilder + ArrayList	CRUD, subranges, sorting, comparator and remove trap.
4	HashMap + HashSet	Frequency, duplicates, first index, grouping, seen-state.
5	Stack + Queue + Deque	Parentheses, simple queue simulation, deque operations.
6	PriorityQueue + TreeMap/TreeSet	Min/max heap, Top-K skeleton, floor/ceiling exercises.
7	Utilities + revision	Comparator, conversions, complexity and a no-notes API test.

Daily rule

Spend roughly **40% learning + 60% typing/practice**. The goal is not to recognize the method when you see it; the goal is to recall it when the problem demands it.

24

One-Page Mental Model

The toolkit in one picture

Need	Reach for	Core Java API
Indexed data	Array / ArrayList	a[i], a.length, get, set
Text	String / StringBuilder	charAt, substring, append, reverse
Count/map	HashMap	put, get, getOrDefault, entrySet
Unique/seen	HashSet	add, contains, remove
LIFO	ArrayDeque	push, peek, pop
FIFO	ArrayDeque	offer, peek, poll
Both ends	Deque / ArrayDeque	offerFirst/Last, pollFirst/Last
Best priority	PriorityQueue	offer, peek, poll
Sorted keys	TreeMap	floorKey, ceilingKey, lowerKey, higherKey
Sorted unique	TreeSet	floor, ceiling, lower, higher
Ordering	Comparator	comparingInt, reversed, thenComparing
Array utilities	Arrays	sort, binarySearch, fill, copyOf
Collection utilities	Collections	sort, reverse, min, max, swap

The DSA mindset

1. Understand the problem. → 2. Identify the pattern. → 3. Choose the structure. → 4. Use the Java API. → 5. Maintain the invariant. → 6. Prove complexity.

Next destination

This toolkit is the **Java foundation layer**. Do not mix it with every DSA pattern at once. Finish the toolkit, then study patterns one at a time — starting with **Two Pointers**.